MODULE 3 // TABLE OF CONTENTS

Linux Operating System

SOC-relevant study notes | 7 documents

01 Linux Systems Overview

01 Linux Systems Overview.pdf

What Linux, the kernel, and distributions are, and why CLI skills matter to analysts.

kernel · open source · distributions · CLI vs GUI

02 Exploring Linux Operating System Essentials

02 Exploring Linux Operating System Essentials.pdf

Why organizations adopt Linux, plus accounts, system info, files, and performance monitoring.

user accounts · system info · file manager · System Monitor

03 Linux Terminal Overview

03 Linux Terminal Overview.pdf

How the terminal and shell work together, and basic directory navigation.

shell vs terminal · prompt · paths · cd ls pwd

04 Overview of Common Linux Shell Commands

04 Overview of Common Linux Shell Commands.pdf

Core command categories and an investigation workflow built from them.

shell commands · logs · archives · networking

05 Linux File Systems and Directory Structure

05 Linux File Systems and Directory Structure.pdf

File types, the filesystem hierarchy, and where to find logs and configuration.

files vs dirs · Linux vs Windows · key directories · where to look

06 Securing Linux

06 Securing Linux.pdf

Hardening measures, their SOC control mapping, and an SSH investigation.

8 measures · control map · SSH scenario · log monitoring

07 Linux Run Levels

07 Linux Run Levels.pdf

Operating states, systemd targets, and investigating reboots and rescue mode.

run levels 0-6 · systemd targets · reboots · rescue mode

LEARNING PATH

01 Overview → 02 Essentials → 03 Terminal → 04 Commands → 05 Filesystem → 06 Security → 07 Run Levels

MODULE 3 // LINUX OPERATING SYSTEMS

Linux Systems Overview

SOC-Relevant Notes | Operating Systems: Overview, Administration, and Security

// ABOUT THESE NOTES

Structured for Notion revision, entry-level SOC Analyst interview preparation, and practical security operations. Based on the provided course transcript, with SOC applications and interview-oriented explanations clearly separated from the source material.

CYAN course content **GREEN** SOC relevance **AMBER** interview point **RED** caution / source note

01 What is Linux?

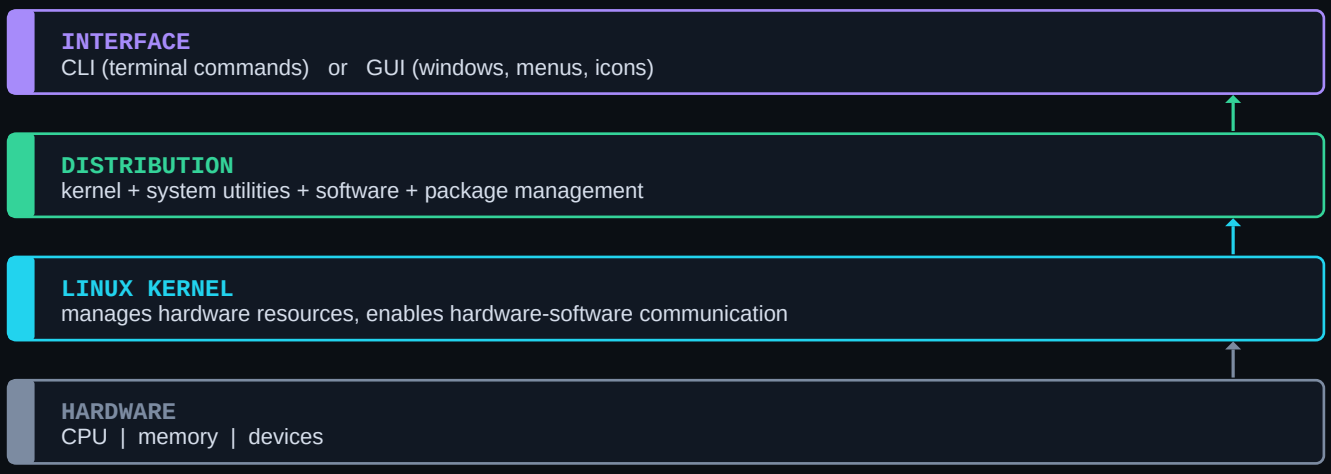
// DEFINITION

Linux is an open-source operating system known for its **stability, security, and versatility**. Its core component, the **Linux kernel**, manages hardware resources and enables communication between hardware and software.

// SOC RELEVANCE

- Linux systems are commonly encountered in **server and infrastructure** environments.
- SOC analysts may investigate suspicious processes, unauthorized account activity, unusual network connections, and changes to system files on Linux hosts.
- Understanding Linux commands helps analysts **collect evidence and investigate incidents** from a terminal.

STACK VIEW how the pieces fit together



02 Open-Source Software

// DEFINITION
Open-source software is distributed under a license that permits users to **view, modify, and distribute** its source code according to the license terms.

KEY CHARACTERISTICS

Transparency
Source code can be inspected.

Collaboration
Developers can contribute improvements and fixes.

Customization
Software can be adapted to specific requirements.

Security review
Public scrutiny can help identify vulnerabilities.

Cost efficiency
It can reduce licensing costs and vendor dependency.

// CAUTION
Reality check
Open-source is **not automatically secure**: configuration, patching, access controls, and monitoring remain essential.

// SOC RELEVANCE
Open-source security tools and Linux utilities can support **log analysis, network investigation, vulnerability assessment, and incident response**.

03 Linux Kernel

// COURSE CONTENT
The kernel is the **foundation** of the Linux operating system. Its responsibilities include:

- Managing CPU, memory, and other hardware resources.
- Supporting communication between hardware and software.
- Providing essential operating system services.
- Supporting the execution of processes and system operations.

// SOC SCENARIO
An analyst investigating suspicious activity on a Linux server may examine **running processes and system behavior** to determine whether an unexpected program is consuming resources or performing unauthorized actions.

04 Popular Linux Distributions

// DEFINITION
A Linux distribution combines the **Linux kernel** with system utilities, software, and package management tools.

DISTRIBUTION	COURSE DESCRIPTION	SOC RELEVANCE
Ubuntu	User-friendly, with extensive community support	Useful for Linux administration and security labs
Fedora	Emphasizes newer software technologies	Useful for understanding modern Linux technologies
CentOS	Described in the course as derived from RHEL source code	Relevant to server administration and enterprise environments
Debian	Focuses on stability, security, and free/open-source principles	Useful for stable server environments
Linux Mint	Provides a user-friendly desktop experience	Useful for learning Linux fundamentals

// SOURCE NOTE
The table reflects the course's descriptions. Linux distributions and their support models can change over time.

// INTERVIEW POINT
A distribution is a **packaged Linux operating system**; the kernel is its core component. Different distributions may use different package managers and administration workflows.

05 CLI vs GUI

$ TERMINAL
Command-Line Interface (CLI)
- Interact with the OS by entering commands in a terminal.
- Efficient for repetitive administrative tasks.
- Supports scripting and automation.
- Uses fewer resources than a GUI in many environments.
- Requires familiarity with commands and syntax.

SOC application: execute commands to inspect processes, investigate files, review logs, and examine network connections.

[] DESKTOP
Graphical User Interface (GUI)
- Interact using windows, menus, icons, and visual controls.
- Easier for beginners to navigate.
- Makes file structures and system resources visually accessible.
- Supports multitasking through windows and tabs.
- Generally consumes more resources than a CLI.

SOC application: review graphical administration tools, security dashboards, and log viewers where available.

INTERVIEW COMPARISON

FEATURE	CLI	GUI
Interaction	Text commands	Visual controls
Automation	Strong scripting support	Often relies on GUI tools or automation frameworks
Resource usage	Usually lower	Usually higher
Learning curve	Requires command knowledge	Generally easier for beginners
SOC use	Investigation and response commands	Dashboards and visual analysis

// INTERVIEW QUESTION + SAMPLE ANSWER

Why is CLI important for a SOC Analyst?

"The command-line interface allows a SOC Analyst to investigate systems efficiently, retrieve relevant evidence, inspect processes, analyze logs, and automate repetitive tasks. Linux servers may operate without a graphical interface, so CLI skills are particularly important during incident investigation."

06 Linux Commands: SOC Preview

// SOURCE NOTE

These commands are practical extensions for SOC learning. They are **not explicitly demonstrated** in the supplied transcript.

COMMAND	PURPOSE	SOC INVESTIGATION EXAMPLE
`whoami`	Displays the current user	Identify the account executing commands
`id`	Displays user and group IDs	Check account privileges and group membership
`ps aux`	Lists running processes	Identify unexpected processes
`top`	Displays live process and resource activity	Investigate unusual CPU or memory usage
`ss -tulpn`	Displays listening sockets and network services	Identify unexpected listening ports; permissions affect process details
`ip addr`	Displays network interfaces and IP addresses	Understand the host's network configuration
`ls -la`	Lists files, including hidden entries	Examine files in a directory
`tail -n 50 /var/log/auth.log`	Displays the last 50 lines of a common authentication log	Review authentication events on systems that use this log path
`grep -i "failed" FILE`	Searches a file for matching text, ignoring case	Search an available log file for failure messages

// IMPORTANT

Linux distributions differ in log locations and logging configuration. For example, authentication events may be recorded in `/var/log/auth.log`, `/var/log/secure`, or the **systemd journal**.

// AUTHORIZATION

Only run investigation commands on systems you **own or are authorized to administer**.

07 SOC Investigation Scenario

// SCENARIO

Suspicious Activity on a Linux Server. A security alert reports unusual activity on a Linux server. Your responsibility is to investigate **before deciding whether to escalate**.

1. **Identify the current account**
   Run `whoami` and `id` to understand the account and its group memberships.
2. **Inspect running processes**
   Run `ps aux` and identify processes that appear unusual or require further investigation.
3. **Review network services**
   Run `ss -tulpn` to examine listening ports and associated processes when available.
4. **Review authentication evidence**
   Examine the relevant authentication logs or system journal for failed logins, unexpected access, or suspicious timing.
5. **Correlate evidence**
   Compare timestamps, account activity, process execution, and network behavior with the alert details.
6. **Make a decision**
   Document findings and determine the outcome (see below).

ESCALATE	CONTAIN	MONITOR	CLOSE
Evidence indicates real risk	Under approved procedures	Continue monitoring	False positive

// CRITICAL-THINKING REMINDER

An unusual process or failed login is **not sufficient by itself** to prove compromise. Validate the evidence, consider legitimate explanations, and follow the organization's incident-response procedures.

08 Interview Revision

// FLASHCARD 01

Q1. What is the Linux kernel?

The kernel is the core component of Linux that manages hardware resources and provides essential services for communication between hardware and software.

// FLASHCARD 02

Q2. What is open-source software?

Software distributed under a license that permits access to, modification of, and redistribution of its source code according to the license terms.

// FLASHCARD 03

Q3. What is the difference between a Linux distribution and the Linux kernel?

The kernel manages core operating system functions. A distribution packages the kernel with system utilities, software, and tools to provide a usable operating system.

// FLASHCARD 04

Q4. Why might a SOC Analyst prefer CLI over GUI?

CLI supports efficient investigation, scripting, automation, and remote administration, including on servers without a graphical desktop.

// FLASHCARD 05

Q5. Does open-source mean software is automatically secure?

No. Source-code transparency can help identify vulnerabilities, but security also depends on patching, secure configuration, access controls, and monitoring.

// FLASHCARD 06

Q6. How does Linux knowledge support incident response?

It helps analysts inspect processes, user accounts, network connections, system logs, and file activity to collect and correlate evidence.

09 Recommended Next Topics

The supplied course outline lists the following upcoming Linux topics. Prioritize them in this order for entry-level SOC preparation:

01	Linux terminal fundamentals and common shell commands
02	Linux user and group management
03	Linux filesystem and directory structure
04	Linux run levels and system operation
05	Securing Linux systems
06	Updating Linux and managing patches
07	Practical investigation exercises and the module quiz

// KEY TAKEAWAY

For a SOC Analyst, understanding Linux concepts is the foundation; the next step is learning how to use **terminal commands** to investigate accounts, processes, authentication events, files, and network activity.

MODULE 3 // LINUX OPERATING SYSTEMS

Exploring Linux OS Essentials

SOC-Relevant Notes | Notion-ready | Entry-level SOC Analyst preparation

01 Why Linux **02** User accounts **03** System info **04** File system **05** Monitoring

CYAN course content **GREEN** SOC relevance **AMBER** interview point **RED** caution / note

01 Why Organizations Adopt Linux

// COURSE CONTENT

Linux is an open-source operating system available through different distributions, or **distros**. Organizations and developers commonly use Linux because of its **stability, security, and efficiency**.

KEY LINUX DISTRIBUTIONS

Ubuntu

Distributed by Canonical; offers a user-friendly graphical interface.

Debian

A widely used Linux distribution.

Red Hat

Provides Linux solutions for enterprise environments.

Linux Mint

Emphasizes ease of use.

openSUSE

Another Linux distribution used for desktop and server environments.

// SOC RELEVANCE

SOC Analysts may investigate Linux workstations, servers, and other systems. Understanding the OS helps analysts investigate **suspicious account activity, running processes, file changes, and resource usage**.

// INTERVIEW QUESTION + SAMPLE ANSWER

Why do organizations use Linux?

"Organizations adopt Linux because of its stability, security, and efficiency. It offers flexibility across desktop and server environments and provides command-line tools that support system administration, automation, and security investigations."

02 Linux Local User Account Management

// DEFINITION
A local user account allows an individual to access a particular system. The course demonstrates creating a **standard user account without administrative rights**.

STEPS: CREATE A STANDARD USER IN UBUNTU GUI

01	Log in using an administrator account.
02	Open Activities and search for `Users` in Settings.
03	Select Users, then click **Unlock**.
04	Enter the administrator password.
05	Click **Add User** and select the **Standard** account type.
06	Enter the user's full name and username.
07	Select the option that requires the user to set a password at their next login, if available.
08	Click **Add** to create the account.
09	Switch users and sign in to the new account, following the password setup prompts.
10	Return to Settings > Users to verify the account information.

// NOTE
Exact labels and navigation may differ by Ubuntu version. The transcript's example password is intentionally not reproduced; use a **unique, secure password** in a lab.

STANDARD USER vs ADMINISTRATOR

STANDARD USER	ADMINISTRATOR
Performs normal tasks within permitted access	Can perform privileged administrative tasks
Has limited system-level permissions	Can manage system configuration and users
Helps reduce the impact of compromised accounts	Requires stronger protection because of elevated privileges

02.1 Least Privilege (SOC Relevance)

// PRINCIPLE
The principle of **least privilege** means granting users only the access necessary to perform their duties.

SOC ANALYSTS MAY INVESTIGATE

// SOC RELEVANCE
- Unexpected account creation.
- Unauthorized changes to account privileges.
- Accounts with unnecessary administrative access.
- Suspicious activity involving privileged accounts.
- Attempts to gain elevated permissions.

// EXAMPLE SCENARIO

A standard user account is unexpectedly granted administrative privileges. A SOC Analyst should verify:

- **who** made the change
- **when** it occurred
- whether it was **authorized**
- what **activity followed**

03 Viewing Linux System Information

// COURSE CONTENT

Ubuntu's **About** settings page displays information about the workstation's hardware and operating system.

STEPS

01	Open Activities.
02	Search for `About`.
03	Open the Settings option for viewing system information.
04	Review the available system details.

INFORMATION AVAILABLE

INFORMATION	WHY IT MATTERS
Device name	Identifies the workstation
Hardware details	Helps establish the system's configuration
Memory and processor	Helps assess system capacity
Graphics information	Identifies graphics hardware
Disk capacity	Helps assess available storage
OS name and type	Identifies the operating system
GNOME version	Identifies the desktop environment version
Windowing system	Shows the graphical display system
Virtualization	Provides information about the virtualized environment, when applicable
Software update status	Helps determine whether updates may be pending

// SOC RELEVANCE

System information provides context during incident investigation. Analysts need to identify the **affected host**, understand its operating environment, and determine whether its configuration or update status is relevant to an alert.

// EXAMPLE

An alert identifies a potentially compromised Ubuntu workstation. Confirming its **device name and operating system** helps correlate the endpoint with security logs and asset inventory records.

04 Ubuntu File System and File Management

// COURSE CONTENT

Ubuntu provides a graphical file manager that lets users navigate folders, create directories, and open or save documents.

COMMON HOME FOLDER DIRECTORIES

Desktop
Desktop files and shortcuts.

Documents
Documents and reports.

Downloads
Downloaded files.

Music
Audio files.

Pictures
Images.

Templates
Reusable file templates.

Videos
Video files.

The exact folders present depend on the system and user configuration.

CREATING A FOLDER AND FILE (COURSE WORKFLOW)

01	Open Files or the Home Folder.
02	Right-click within the desired location and select **New Folder**.
03	Name the folder `Test Folder 1`.
04	Open the new folder.
05	Create a document using **LibreOffice**.
06	Enter sample text and save the file in the new folder.
07	Reopen the folder to verify that the file exists.

SOC RELEVANCE: FILE INVESTIGATION

// ANALYSTS MAY EXAMINE

- Unexpected files in user directories.
- Suspicious files in `Downloads` or temporary directories.
- Unusual file creation or modification times.
- Unauthorized changes to important configuration files.
- Potentially malicious scripts or executables.

// EXAMPLE SCENARIO

An endpoint alert reports that a script was downloaded and executed. An analyst can examine the relevant user directory, determine whether the file is present, **review its metadata**, and correlate the findings with process and authentication logs.

// IMPORTANT
A suspicious filename or file location **alone does not establish** that a file is malicious. Validate it using additional evidence.

05 Monitoring Linux System Performance

// COURSE CONTENT
Ubuntu's **System Monitor** application provides information about processes, resource usage, and file systems.

HOW TO OPEN SYSTEM MONITOR

01	Open Activities.
02	Search for `System`.
03	Select **System Monitor**.
04	Review the available tabs.

SYSTEM MONITOR TABS

TAB	INFORMATION	SOC RELEVANCE
Processes	Running applications and processes	Investigate unexpected or resource-intensive processes
Resources	CPU, memory, and other resource activity	Identify unusual resource consumption
File Systems	Storage and filesystem usage	Investigate storage exhaustion or unusual disk usage

SOC INVESTIGATION EXAMPLES

// SCENARIO 1
Unusually high CPU usage
A Linux server is consuming significantly more CPU than expected.
- Review the **Processes** tab.
- Identify processes consuming substantial CPU resources.
- Check whether the activity matches expected workloads.
- Correlate the process with relevant logs and security alerts.

// SCENARIO 2
High memory usage
A server becomes slow or unresponsive.
- Review memory utilization in **Resources**.
- Identify processes with unusually high memory consumption.
- Determine whether the behavior is expected or suspicious.
- Document findings for further investigation.

// SCENARIO 3
Unusual disk usage
A server's storage is nearly full.
- Review the **File Systems** tab.
- Identify which filesystems are nearing capacity.
- Investigate relevant directories and logs.
- Determine whether the cause is legitimate workload growth, excessive logging, or potentially malicious activity.

// SECURITY CAUTION

High CPU, memory, or disk usage can indicate a **normal workload, a software defect, a denial-of-service condition, or malicious activity**. Performance anomalies require investigation rather than an immediate conclusion of compromise.

06 SOC Interview Questions and Answers

// FLASHCARD 01

Q1. How do you create a standard user in Ubuntu?

Log in with an administrator account, open Settings > Users, unlock the settings, select Add User, choose Standard, enter the account details, and verify the account after creation.

// FLASHCARD 02

Q2. Why should users avoid unnecessary administrative privileges?

Least privilege reduces the potential impact of compromised accounts and limits unauthorized system changes.

// FLASHCARD 03

Q3. How can system information help during incident response?

It helps identify the affected device, establish its operating system and configuration, and correlate the host with alerts and asset records.

// FLASHCARD 04

Q4. How would you investigate a suspicious process in Linux?

I would identify the process, examine its resource usage and context, correlate it with relevant system and security logs, and validate whether its behavior is authorized before recommending further action.

// FLASHCARD 05

Q5. What is the difference between system performance monitoring and security monitoring?

Performance monitoring focuses on resource utilization and system health. Security monitoring focuses on detecting suspicious or unauthorized activity. Performance anomalies can provide clues for a security investigation, but they do not independently prove malicious activity.

07 Quick Revision Checklist

REVISION PROGRESS — **0 of 8 completed**

- [ ] Explain why organizations adopt Linux.
- [ ] Identify common Linux distributions.
- [ ] Describe the steps to create and verify a standard user account.
- [ ] Explain the principle of least privilege.
- [ ] Identify the information shown in Ubuntu About settings.
- [ ] Describe the Ubuntu Home Folder and basic file operations.
- [ ] Explain the Processes, Resources, and File Systems tabs.
- [ ] Describe how to investigate an unusual process or resource spike.

// KEY TAKEAWAY

For an entry-level SOC Analyst, this lesson establishes four practical foundations: **Linux account management, host identification, file-system navigation, and performance monitoring**. The next step is learning the Linux terminal and using commands to investigate authentication events, processes, files, and network activity.

MODULE 3 // LINUX OPERATING SYSTEMS

Linux Terminal Overview

SOC-Relevant Notes | Visual edition | Entry-level SOC Analyst preparation

```
analyst@soc-lab: ~
analyst@soc-lab:~$ whoami
analyst
analyst@soc-lab:~$ pwd
/home/analyst
analyst@soc-lab:~$
```

SHELL
interprets commands

TERMINAL
input and output

PATHS
absolute and relative

NAVIGATION
cd ls pwd

SOC USE
investigate hosts

CYAN course content **GREEN** SOC relevance / code **AMBER** interview point **RED** caution

01 What Is a Linux Shell?

// DEFINITION

A Linux shell is an operating-system-level application that **interprets commands** and enables users to interact with the operating system.

SHELLS CAN EXECUTE COMMANDS AND SCRIPTS TO PERFORM TASKS SUCH AS

FILES
Create, move, copy, and delete files.

CONTENTS
Read and write file contents.

FILTER
Extract and filter data.

SEARCH
Search for specific information.

AUTOMATE
Automate repetitive tasks through scripts.

COMMON LINUX SHELLS

Bash (Bourne Again Shell)

A widely used shell for Linux commands and scripting.

Zsh (Z Shell)

A shell with interactive features and scripting capabilities.

// SOC RELEVANCE

SOC Analysts use shell commands to **investigate Linux systems, search logs, inspect suspicious files, and automate repetitive investigation tasks**.

02 What Is a Linux Terminal?

// DEFINITION

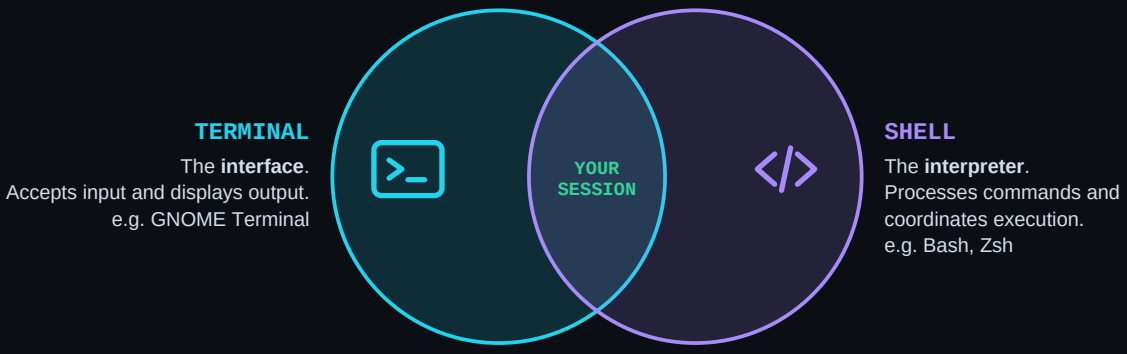
A terminal is an application or user interface where you **enter commands and view their output**. The terminal provides access to a shell, which interprets and executes the commands.

EXAMPLE

```
$ python myprogram.py
Hello, World!
```

This command asks Python to run a program named `myprogram.py`. The program's output, such as `Hello, World!`, appears in the terminal if the program executes successfully and prints that message.

TERMINAL vs SHELL



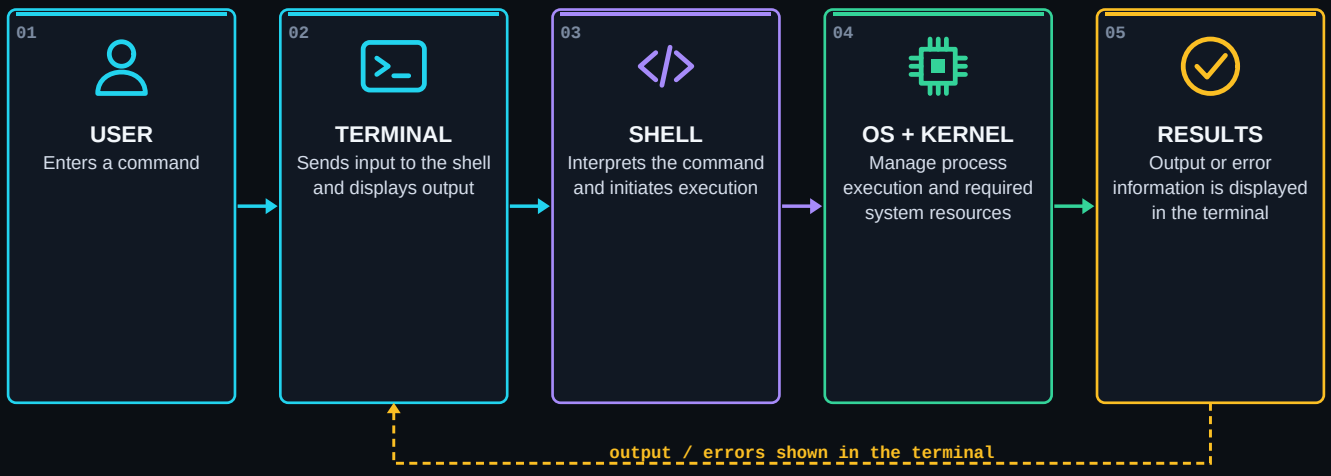
FEATURE	TERMINAL	SHELL
Definition	Application or interface for entering commands	Program that interprets commands
Main responsibility	Accepts input and displays output	Processes commands and coordinates their execution
Examples	GNOME Terminal	Bash, Zsh
SOC use	Provides the command-line workspace	Executes investigation commands and scripts

// INTERVIEW QUESTION + SAMPLE ANSWER

What is the difference between a terminal and a shell?

"A terminal is the interface where a user enters commands and sees the results. The shell interprets those commands and coordinates their execution. For example, I can use GNOME Terminal to interact with Bash while investigating a Linux host."

03 How Terminal and Shell Work Together



// SOC RELEVANCE

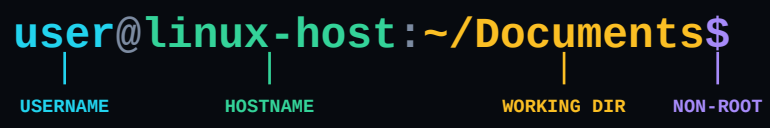
Commands execute with the **permissions of the current user** unless additional privileges are explicitly obtained. Commands that modify system settings may require elevated privileges.

04 Understanding the Linux Command Prompt

// COURSE CONTENT

The **command line** is the area where commands are entered. The cursor indicates where the next characters will appear. A terminal prompt may display the current username, hostname, and working directory; its exact format depends on the shell configuration.

```
user@linux-host:~/Documents$
```



Exact prompt format depends on the shell configuration.

PART	MEANING
`user`	username
`linux-host`	hostname
`~/Documents`	current working directory
`$`	commonly indicates a non-root shell prompt

// NOTE

A # prompt is commonly associated with a **root shell**, although prompt symbols can be customized.

CURRENT WORKING DIRECTORY (CWD)

// DEFINITION

The CWD is the directory in which the shell is currently operating. If the CWD is `/home/user/Documents`, a relative filename such as `report.txt` refers to that directory unless another path is specified.

// SOC RELEVANCE

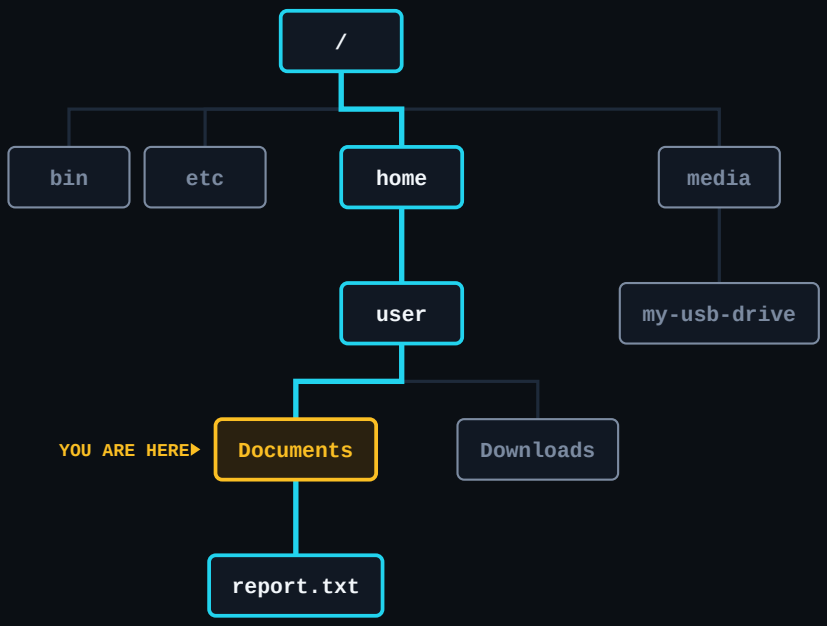
Knowing the current directory helps prevent analysts from **inspecting the wrong files or running commands against an unintended location**.

05 Linux File Paths and Special Symbols

// DEFINITION

A **path** identifies the location of a file or directory within the filesystem.

FILESYSTEM MAP: ONE LOCATION, TWO WAYS TO WRITE IT



FROM CWD = Documents

`.`
/home/user/Documents

`..`
/home/user

`../..`
/home

`~`
/home/user

`/`
filesystem root

ABSOLUTE `/home/user/Documents/report.txt`

RELATIVE `report.txt` (from the Documents directory)

Absolute path

Specifies a location starting from the root directory, such as `/home/user/Documents`.

Relative path

Specifies a location relative to the current working directory, such as `Documents/report.txt`.

SPECIAL SYMBOLS

/

ROOT

Root directory at the start of a path

`/home/user`

~

HOME

Current user's home directory

`cd ~`

.

CURRENT

Current directory

`./script.sh`

. .

PARENT

Parent directory

`cd ..`

./

REL PATH

Path relative to the current directory

`./program`

SYMBOL	MEANING	EXAMPLE
`/`	Root directory when used at the beginning of a path	`/home/user`
`~`	Current user's home directory	`cd ~`
`.`	Current directory	`./script.sh`
`..`	Parent directory	`cd ..`
`./`	A path relative to the current directory	`./program`

`// IMPORTANT DISTINCTION`

`/` represents the **filesystem root**, whereas `/home/user` is a directory within that root.





06 Essential Directory Navigation Commands

`// COURSE CONTENT`

These are the key commands demonstrated or directly supported by the course.



COMMAND	PURPOSE	EXAMPLE
`cd /`	Navigate to the root directory	`cd /`
`cd ~`	Navigate to the current user's home directory	`cd ~`
`cd ..`	Move to the parent directory	`cd ..`
`cd bin`	Enter the bin directory inside the current directory	`cd bin`
`cd /media/my-usb-drive`	Navigate to a USB drive's mount path in the example	`cd /media/my-usb-drive`
`cd ../..`	Move up two directory levels	`cd ../..`
`ls`	List files and directories	`ls`
`./ls`	Attempt to execute a file named ls in the current directory	`./ls`

UNDERSTANDING ls AND ./ls

The course demonstrates executing `./ls` from `/bin` and using `ls` from another directory. The distinction is important:

LANE A // ls

01
ls
Typed without an explicit path

02
SHELL
Uses its command lookup mechanism

03
PATH SEARCH
Commonly uses the PATH environment variable

04
RUN
Executes the command that was found

LANE B // ./ls

01
./ls
Explicit reference to the current directory

02
CURRENT DIR
Refers to a file named ls right here

03
CHECK FILE
Executable file must exist with execution permission

04
RUN
Executes only if both conditions are met

// NOTE
On modern Linux distributions the exact location of the `ls` executable can vary; `/bin` may also be linked to `/usr/bin`.

07 Hands-On Lab: Practice Linux Navigation

// BEGINNER LAB | READ-ONLY NAVIGATION
Run the following commands in your Linux terminal. Press **Enter** after each command.

1. `pwd` — where am I
2. `ls` — what is here
3. `cd ~` — go home
4. `cd ..` — go up
5. `cd Documents` — go in

STEP 1

Check your current directory

```
$ pwd
```

Expected result: An absolute path showing your current working directory.

STEP 2

List files and directories

```
$ ls
```

Expected result: The names of entries in the current directory, depending on its contents.

STEP 3

Navigate to your home directory

```
$ cd ~
```

Verify:

```
$ pwd
```

Expected result: Your user's home directory, typically `/home/yourusername` on a standard Ubuntu installation.

STEP 4

Move to the parent directory

```
$ cd ..
```

Verify:

```
$ pwd
```

Expected result: The parent of your home directory, typically `/home`.

STEP 5

Return to your home directory, then enter Documents if it exists

```
$ cd ~
$ cd Documents
```

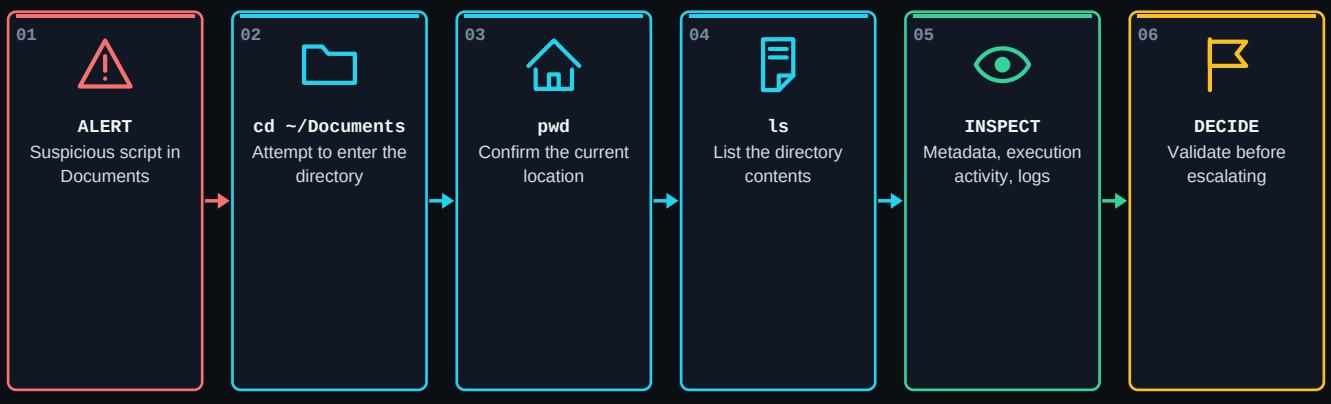
Expected result: You enter the Documents directory if it exists under your home directory.

08 SOC Investigation Scenario

// SCENARIO

A suspicious script is discovered on a Linux endpoint. A monitoring tool generates an alert about a potentially suspicious script in a user's Documents directory. A SOC Analyst can use basic navigation to locate the directory and inspect its contents.

INVESTIGATION FLOW




```
cd ~/Documents
pwd
ls
```

WHAT EACH COMMAND ACCOMPLISHES

01	`cd ~/Documents`: attempts to enter the Documents directory.
02	`pwd`: confirms the current location.
03	`ls`: lists the directory's contents.

// NEXT

The analyst can then proceed to **authorized evidence collection** and inspect relevant file metadata, execution activity, and security logs.

// CRITICAL-THINKING POINT

Finding a script in a user's directory **does not prove malicious activity**. Determine its origin, purpose, execution history, and relationship to the alert before deciding whether to escalate.

09 SOC Interview Questions

// FLASHCARD 01

Q1. What is a Linux shell?

A shell is a program that interprets user commands and coordinates their execution by the operating system.

// FLASHCARD 02

Q2. What is a Linux terminal?

A terminal is an application or interface through which a user enters commands and views their output.

// FLASHCARD 03

Q3. What is the difference between `.` and `..`?

`.` represents the current directory, while `..` represents the parent directory.

// FLASHCARD 04

Q4. What is the difference between `cd /` and `cd ~`?

`cd /` navigates to the filesystem root. `cd ~` navigates to the current user's home directory.

// FLASHCARD 05

Q5. What is the difference between an absolute path and a relative path?

An absolute path identifies a location from the filesystem root. A relative path identifies a location in relation to the current working directory.

// FLASHCARD 06

Q6. Why should a SOC Analyst understand Linux directory navigation?

It helps the analyst locate evidence, inspect relevant files, confirm the working directory, and operate accurately during host investigations.

10 Quick Revision Checklist

LEARNING PROGRESS 0 / 8

- [ ] Explain the difference between a shell and a terminal.
- [ ] Describe how a command is executed.
- [ ] Identify Bash and Zsh as Linux shells.
- [ ] Explain the current working directory.
- [ ] Differentiate absolute and relative paths.
- [ ] Explain the meanings of `/`, `~`, `.`, and `..`.
- [ ] Practice `cd`, `ls`, and `pwd` commands.
- [ ] Explain how directory navigation supports a SOC investigation.



TERMINAL
The interface: where you type and read output



SHELL
The interpreter: runs your commands



NAVIGATION
Move through the Linux filesystem

`// KEY TAKEAWAY`

The **terminal** provides the interface, the **shell** interprets commands, and **directory navigation** lets you move through the Linux filesystem. These are foundational skills for learning Linux log analysis, process investigation, and incident response as an entry-level SOC Analyst.

Common Linux Shell Commands

SOC-Relevant Notes | Visual edition | Overview of Common Linux Shell Commands

```
analyst@soc-lab: ~
analyst@soc-lab:~$ whoami
analyst
analyst@soc-lab:~$ pwd
/home/analyst
analyst@soc-lab:~$
```

A SYSTEM	B FILES	C DIRS	D DISPLAY	E ARCHIVE	F NETWORK
whoami id uname ps	cp mv rm chmod	ls find cd pwd	cat head tail	tar zip unzip	ping curl wget

CYAN course content **GREEN** SOC relevance / code **AMBER** interview point **RED** caution

01 What Is a Linux Shell?

// DEFINITION

A shell is a user interface for Unix-like operating systems that **interprets commands and runs programs**. It also provides a **scripting language** for automating tasks. The shell enables users to interact with files, utilities, applications, and system resources.

COMMON LINUX SHELLS

BASH	sh	ksh	tcsh	zsh	fish
Bourne Again Shell; the shell used throughout the course	SH Bourne shell	KSH Korn shell	TCSH An enhanced C shell	ZSH Z shell	FISH Friendly Interactive Shell

IDENTIFY YOUR DEFAULT SHELL

```
$ printenv SHELL
/bin/bash
```

Displays the configured default shell's path, for example `/bin/bash`. To start Bash from your current terminal session, enter:

```
$ bash
```

// SOC RELEVANCE

Shell knowledge helps analysts **execute investigation commands, inspect system activity, search logs, and automate repetitive tasks**.

02 Common Linux Commands for SOC Analysts

// COURSE CONTENT
The course groups shell commands by their primary application. The tables below preserve those categories and highlight their potential SOC use.

CATEGORY MAP

A SYSTEM	B FILES	C DIRS	D DISPLAY	E ARCHIVE	F NETWORK
whoami id uname ps	cp mv rm chmod	ls find cd pwd	cat head tail	tar zip unzip	ping curl wget

A System and User Information

COMMAND	PURPOSE	SOC APPLICATION
`whoami`	Displays the current username	Identify the account running commands
`id`	Displays user and group IDs	Investigate account privileges and group membership
`uname`	Displays operating system information	Identify system and kernel details
`ps`	Displays running processes and their IDs	Investigate suspicious processes
`top`	Displays processes and resource usage, including CPU and memory	Investigate unusual resource consumption
`df`	Displays information about mounted filesystems	Check disk utilization
`man`	Opens a command's reference manual	Learn command syntax and options
`date`	Displays the system date and time	Understand timestamps during investigations

EXAMPLES

```
whoami
id
uname -a
ps
top
df -h
man ps
date
```

// SOC SCENARIO
An alert identifies suspicious activity on a Linux endpoint. Start by checking the **current user, system details, running processes, and resource usage** to establish the host's context.

// NOTE

`date` displays the system clock; **verify time synchronization and timezone** when correlating events across systems.

`ps` = snapshot

Displays process information **at the time it is executed**.

`top` = live view

Provides a **continuously updating** view of processes and system resource usage.

B File Operations and Permissions

COMMAND	PURPOSE	SOC APPLICATION
`cp`	Copies files	Preserve working copies of authorized evidence
`mv`	Moves or renames files	Understand file movement during investigations
`rm`	Removes files	Investigate possible evidence deletion
`touch`	Creates an empty file or updates timestamps	Understand file creation and timestamp changes
`chmod`	Changes file permissions	Investigate suspicious permission changes
`wc`	Counts lines, words, and bytes or characters	Estimate log size and count records
`grep`	Searches text for matching patterns	Find suspicious messages in logs

EXAMPLES

```
cp report.txt report-copy.txt
mv old-name.txt new-name.txt
touch test.txt
chmod 640 report.txt
wc -l /var/log/syslog
grep -i "failed" /var/log/syslog
```

// SOC SCENARIO

An analyst finds a script with unexpected permissions. The analyst can inspect its **permissions, ownership, metadata, and execution history** to determine whether the change was authorized.

// IMPORTANT

`rm`, `mv`, `touch`, and `chmod` **can change evidence or system state**. Avoid using modifying commands on original evidence unless explicitly authorized and covered by evidence-handling procedures.

C Directory Navigation and Management

COMMAND	PURPOSE	SOC APPLICATION
`ls`	Lists files and directories	Locate potentially suspicious files
`find`	Searches directory trees for matching files	Locate files by name or other criteria
`pwd`	Displays the current working directory	Confirm the current investigation location
`mkdir`	Creates a directory	Organize authorized working files
`cd`	Changes the current directory	Navigate to relevant log or evidence directories
`rmdir`	Removes an empty directory	Understand directory-removal activity

EXAMPLES

```
pwd
ls -la
cd /var/log
find /home -type f -name "*.sh" 2>/dev/null
mkdir investigation-notes
```

ANATOMY OF THE find EXAMPLE

The `find` example searches `/home` for files ending in `.sh`. The `2>/dev/null` portion suppresses error messages, such as some permission-denied messages, so it can also **hide useful investigation context**.



Discarded errors can also hide useful investigation context.

```
// SOC SCENARIO
```
A security alert identifies a suspicious shell script. Use `find` to locate matching files in **authorized directories**, then examine the relevant metadata and logs.

D Displaying File Contents and Text

COMMAND	PURPOSE	SOC APPLICATION
`cat`	Displays file contents	Read small configuration or log files
`more`	Displays file contents one page at a time	Review longer files
`head`	Displays the first N lines	Inspect the beginning of a log
`tail`	Displays the last N lines	Review recent log entries
`echo`	Prints a string or variable value	Display variables or test simple shell expressions

EXAMPLES

```
cat /etc/hostname
head -n 20 /var/log/syslog
tail -n 50 /var/log/syslog
echo "SOC investigation started"
echo "$SHELL"
```

TO FOLLOW A LOG AS NEW LINES ARE APPENDED

```
tail -f /var/log/syslog
```

// SOC SCENARIO

After receiving an alert about failed authentication attempts, an analyst can inspect relevant log entries and **correlate timestamps** with the alert.



// REMEMBER

Log filenames and locations depend on the distribution and logging configuration. On some Ubuntu systems, authentication events may be available in `/var/log/auth.log` or through `journalctl`.



E File Compression and Archiving

COMMAND	PURPOSE	SOC APPLICATION
`tar`	Archives a collection of files	Package authorized investigation files
`zip`	Compresses files into a ZIP archive	Package reports and evidence copies
`unzip`	Extracts files from ZIP archives	Inspect authorized archived evidence

EXAMPLES

```
tar -cf logs.tar logs/
zip logs.zip report.txt
unzip logs.zip
```

// SOC RELEVANCE

Analysts may encounter compressed files during investigations, **preserve collections of logs**, or review archives associated with suspicious activity.



// SECURITY NOTE

Inspect untrusted archives cautiously. Extraction can **overwrite files, consume substantial disk space, or expose the system to malicious content**.



F Networking Commands

COMMAND	PURPOSE	SOC APPLICATION
`hostname`	Displays the system's hostname	Identify the affected host
`ping`	Sends ICMP echo requests and displays responses, when supported	Test basic network reachability
`ifconfig`	Displays or configures network interfaces	Inspect interface configuration on systems where available
`curl`	Transfers data to or from a URL	Retrieve or inspect remote content
`wget`	Downloads files from URLs	Download authorized files or security resources

EXAMPLES

```
hostname
ping -c 4 example.com
ifconfig
curl -I https://example.com
wget https://example.com/file.txt
```

// COURSE CONTENT

Some modern Linux distributions do not install `ifconfig` by default. The `ip addr` command is a common alternative for inspecting network interfaces.

// SOC SCENARIO

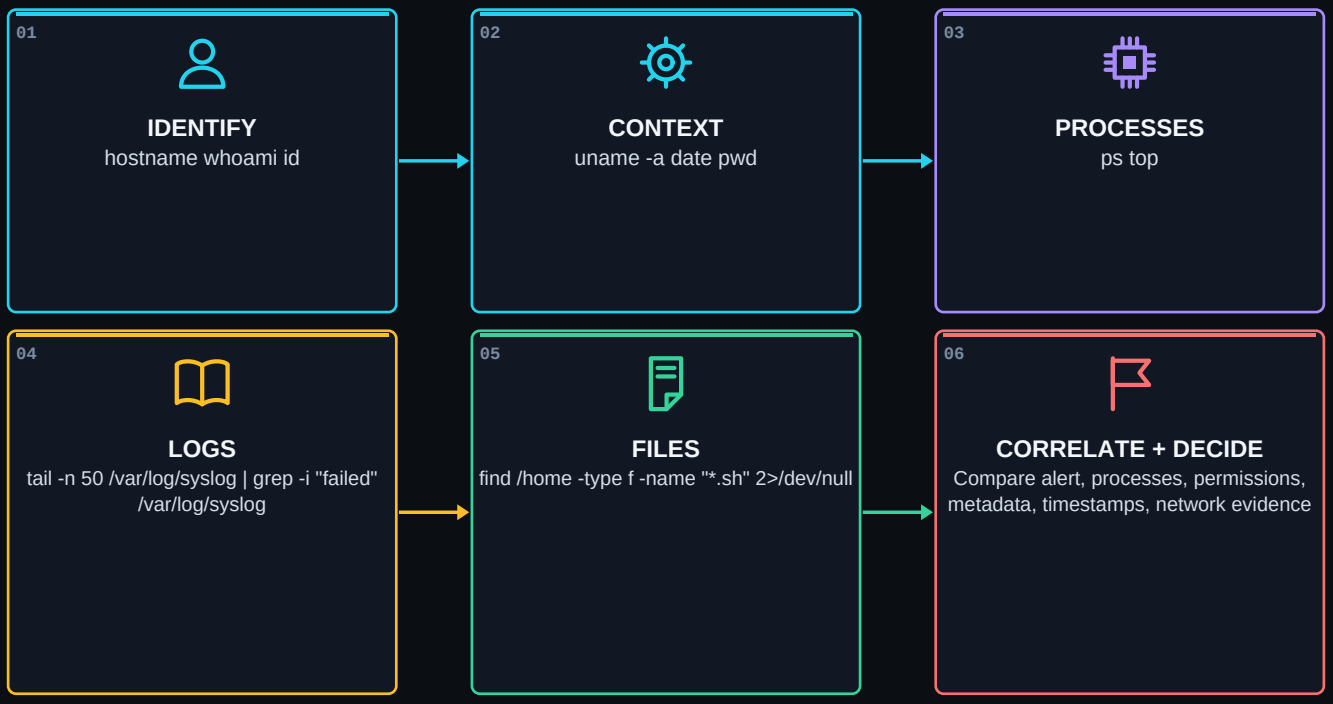
An alert reports suspicious outbound connections. The analyst can confirm the host's identity, inspect network configuration, and investigate the relevant destination using **approved network and endpoint telemetry**.

// SAFETY REMINDER

`curl` and `wget` can contact external systems or retrieve potentially malicious content. Only access destinations and download files in **accordance with organizational policy**.

03 SOC Investigation Workflow Using Linux Commands

// NOTE
This workflow is an illustrative application of the commands in your course, **not a prescribed sequence from the video**.



// IMPORTANT
Confirm the appropriate **log path** before running the log commands. Escalate or respond according to **approved procedures**.

COMMAND MAP: WHAT EACH STAGE TELLS YOU

STEP	COMMANDS	ANSWERS
1 Identify	`hostname`, `whoami`, `id`	Which host? Which account? What privileges?
2 Context	`uname -a`, `date`, `pwd`	What system, what time, where am I?
3 Processes	`ps`, `top`	What is running and consuming resources?
4 Logs	`tail`, `grep`	What happened, and when?
5 Files	`find`	Where are the suspicious files?
6 Decide	evidence review	Escalate, respond, or close

04 Running Linux on Windows

// COURSE CONTENT

The course describes several ways to use Linux on a Windows machine:

DUAL BOOT
Install Linux on a separate partition and reboot to switch operating systems.

VIRTUAL MACHINE
Run Linux inside a virtualized environment.

CYGWIN
Provide a Unix-like environment and tools on Windows.

WSL
Windows Subsystem for Linux: run Linux distributions and supported Linux binaries within Windows.

// SOC RELEVANCE

For SOC learning, a Linux **virtual machine** or **WSL** environment can provide a useful place to practise shell commands. A virtual machine is particularly useful when you want an **isolated lab** with its own operating system.

05 SOC Interview Questions and Answers

// FLASHCARD 01

Q1. What is Bash?

Bash stands for Bourne Again Shell. It interprets commands and supports scripting and automation.

// FLASHCARD 02

Q2. What is the difference between `ps` and `top`?

`ps` displays process information at the time it is executed. `top` provides a continuously updating view of processes and system resource usage.

// FLASHCARD 03

Q3. How would you search a Linux log for failed events?

I would use `grep` to search the relevant log for failure-related patterns, then correlate the results with timestamps, accounts, source addresses, and other available evidence.

// FLASHCARD 04

Q4. What is the purpose of `chmod`?

`chmod` changes file permissions. During a security investigation, unexpected permission changes may warrant further investigation.

// FLASHCARD 05

Q5. What is the difference between `curl` and `wget`?

Both can transfer data over network protocols. `curl` supports a wide range of transfer operations and is frequently used to inspect HTTP responses, while `wget` is commonly used to download files.

// FLASHCARD 06

Q6. Why are `grep`, `tail`, and `find` useful to a SOC Analyst?

`grep` searches text for patterns, `tail` displays the end of a file or follows new entries, and `find` locates files within directory trees. Together, they support basic log and filesystem investigation.

06 Quick Revision Checklist

REVISION PROGRESS 0 of 8

- [ ] Explain what a shell is and identify Bash.
- [ ] Use `printenv SHELL` to inspect the default shell setting.
- [ ] Identify commands for system and user information.
- [ ] Distinguish file operations from directory operations.
- [ ] Use `grep`, `cat`, `head`, and `tail` to inspect text.
- [ ] Explain `tar`, `zip`, and `unzip`.
- [ ] Identify the purposes of `hostname`, `ping`, `curl`, and `wget`.
- [ ] Explain how Linux commands support SOC investigations.

FOCUS FIRST: THE 11 COMMANDS THAT MATTER MOST

ACCOUNT	`whoami` `id`
PROCESSES	`ps` `top`
FILES	`ls` `find`
LOGS	`grep` `tail`
PERMISSIONS	`chmod`
NETWORK CONTEXT	`hostname`
SYSTEM HEALTH	`df`

// KEY TAKEAWAY

For SOC work, focus first on `whoami`, `id`, `ps`, `top`, `ls`, `find`, `grep`, `tail`, `chmod`, `hostname`, and `df`. These commands build the foundation for investigating **accounts, processes, logs, files, network context, and system health**.

Linux File Systems and Directory Structure

SOC-Relevant Notes | Visual edition | Study notes

```
warren@linux-host: ~
$ ls -l
-rw-r--r-- 1 user user 1200 notes.txt
drwxr-xr-x 2 user user 4096 Documents
$
```

FILES
regular files: -

DIRECTORIES
directories: d

HIERARCHY
one tree from /

KEY DIRS
bin sbin etc var

SOC USE
where to look

CYAN course content **GREEN** SOC relevance / code **AMBER** interview point **RED** caution

01 Learning Objectives

// COURSE CONTENT
After studying this topic, you should be able to:

DEFINE
Define files and directories in Linux.

COMPARE
Explain the Linux filesystem hierarchy and compare it with Windows.

IDENTIFY
Identify important Linux directories and explain their purpose.

LOCATE
Recognize where SOC analysts can find logs, configurations, executables, and user files.

02 Files vs. Directories

// COURSE CONTENT
Linux uses files and directories as the **fundamental building blocks** for organizing data.

FILE [-]
Stores data, such as text, logs, configuration settings, scripts, or executable programs.

In a Linux directory listing, a regular file is represented by `-`.

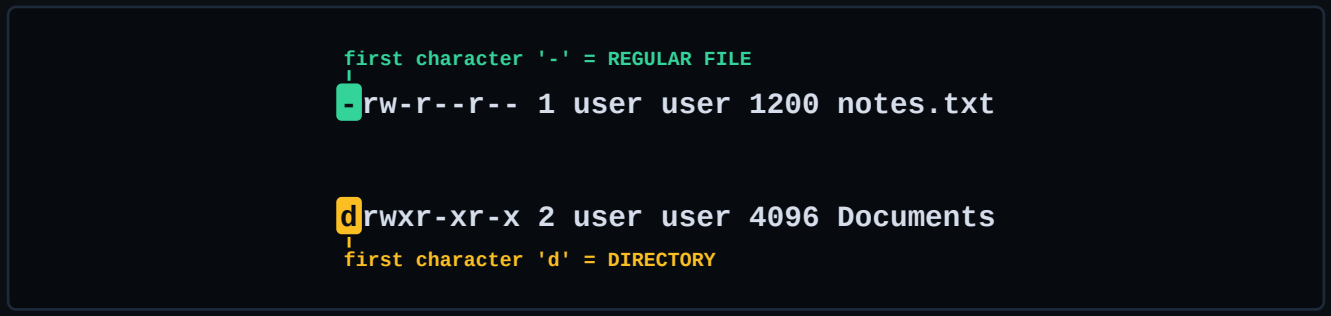
DIRECTORY [d]
Organizes files and other directories, creating a hierarchical structure.

In a Linux directory listing, a directory is represented by `d`.

PRACTICAL COMMAND

```
$ ls -l
```

```
-rw-r--r-- 1 user user 1200 notes.txt
drwxr-xr-x 2 user user 4096 Documents
```

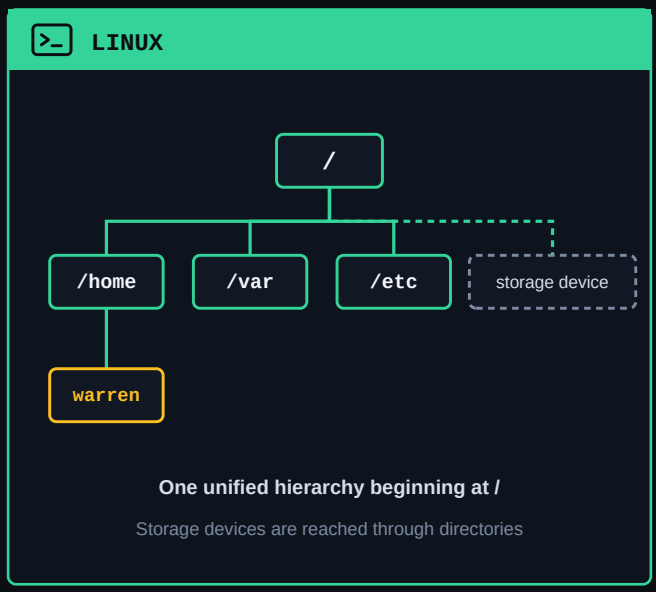


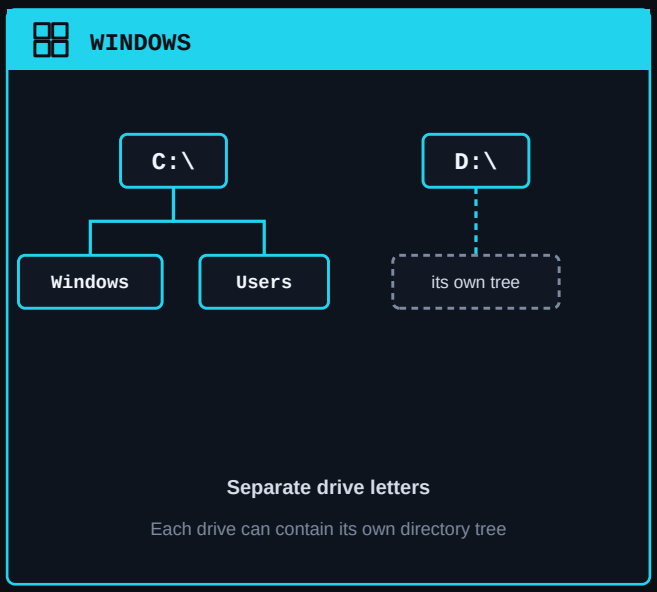
`-rw-r--r--`: the first character `-` indicates a **regular file**.

`drwxr-xr-x`: the first character `d` indicates a **directory**.

// SOC RELEVANCE
Recognizing file types helps analysts **inspect suspicious scripts, log files, directories, and other artifacts** during host investigations.

03 Linux vs. Windows Filesystem Structure





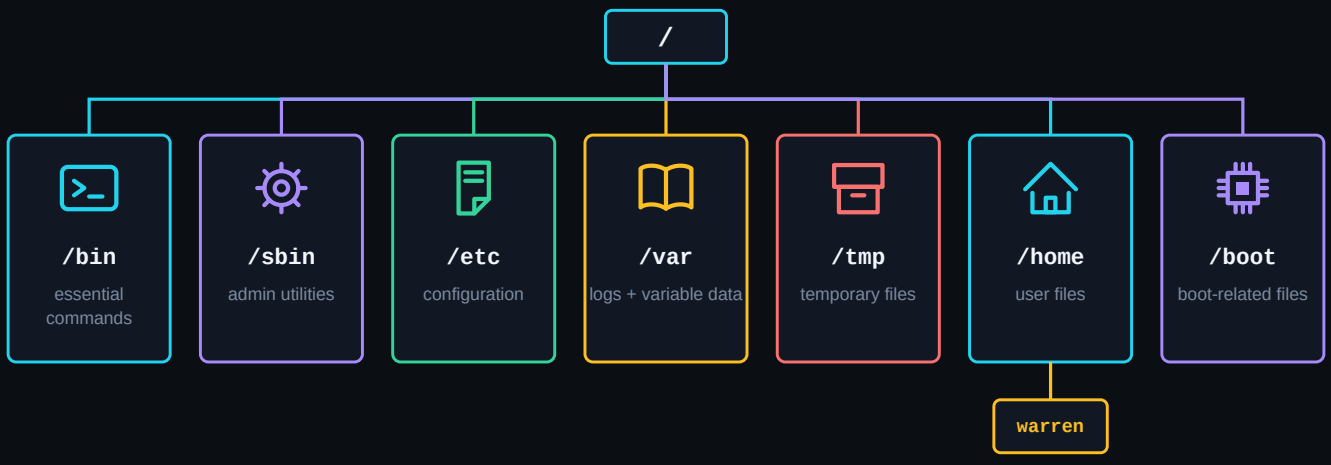
Linux

- One unified hierarchy beginning at `/`.
- Filesystems and storage devices are accessed through directories in this hierarchy.
- Examples: `/home`, `/var`, `/etc`.

Windows

- Commonly uses separate drive letters such as `C:\` and `D:\`.
- Each drive can contain its own directory tree.
- Examples: `C:\Windows`, `C:\Users`.

LINUX DIRECTORY TREE



The `/` directory is the **root** of the Linux filesystem hierarchy. Other directories branch from it.

IMPORTANT DISTINCTION: / vs /root

/
Root of the entire filesystem hierarchy.

/root
Home directory for the root administrator account.

/home/warren
Example of a regular user's home directory.

// IMPORTANT CORRECTION FOR TECHNICAL ACCURACY

`/` is **not necessarily a single disk partition**. It is the top-level directory of the hierarchy, and other filesystems can be mounted at locations beneath it. Also, the root account does not have exclusive permission to modify every item under `/`; **permissions and security controls determine access**.

04 Key Linux Directories

// COURSE CONTENT

These directories are especially useful to understand for Linux administration and SOC investigations.

QUICK REFERENCE: WHERE TO LOOK

LOGS	`/var/log`
CONFIGURATION	`/etc`
TEMPORARY FILES	`/tmp`
USER FILES	`/home`
BOOT-RELATED	`/boot`
COMMANDS	`/bin` `/sbin`

// DIRECTORY

`/bin` Essential user commands

Contains essential executable commands. The transcript lists `ls`, `ps`, `ping`, `grep`, `cp`, and `mv`.

SOC use: Run commands to inspect processes, search logs, and navigate files. The exact contents vary across distributions; some systems link `/bin` to `/usr/bin`.

// DIRECTORY

`/sbin` System administration utilities

The transcript lists `reboot`, `fdisk`, `iptables`, and `ifconfig`.

SOC use: Understand system-maintenance and networking utilities. Some commands require elevated privileges, and their locations vary by distribution.

// DIRECTORY

`/etc` Configuration files

Stores system-wide and application configuration. The transcript gives `/etc/apache` as an example, although the actual Apache configuration path depends on the distribution.

SOC use: Inspect authentication, service, network, and application settings for **unauthorized changes**.

// DIRECTORY

`/var` Variable data

Contains data that changes during system operation, including logs and application data. Common log location: `/var/log`.

SOC use: Investigate authentication events, application activity, and service errors.

// DIRECTORY

/tmp Temporary files

Stores temporary data used by applications and processes. Its contents are not guaranteed to be removed at every reboot; cleanup depends on system configuration.

SOC use: Check for suspicious scripts, temporary payloads, and files created during an incident. **Do not assume every file here is malicious.**

// DIRECTORY

/home Regular user files

Contains home directories for regular users, such as `/home/warren`.

SOC use: Investigate suspicious downloads, scripts, persistence-related files, and unusual user activity while respecting access controls.

// DIRECTORY

/boot Boot-related files

Stores files needed to start the system, often including the Linux kernel and bootloader files.

SOC use: Investigate suspicious changes to boot-related components when examining possible boot-level persistence or system compromise.

05 SOC Investigation: Which Directory Should You Check?

// SCENARIO

Example scenario: an alert reports that a Linux web server may be compromised.

01

CHECK /var/log

Review relevant authentication, system, and web-server logs for suspicious activity.

02

INSPECT /etc

Review relevant service configuration for unexpected changes.

03

INVESTIGATE /tmp

Look for suspicious temporary scripts or payloads associated with the alert.

04

REVIEW /home

Examine relevant user files and activity for signs of compromise.

05

ASSESS /boot (IF INDICATED)

Consider boot-related artifacts if evidence suggests deeper persistence or tampering.

// INVESTIGATION PRINCIPLE

Preserve evidence, establish timestamps, correlate findings with process and authentication events, and follow your organization's incident-response procedures.

06 Hands-On Linux Lab

```
// BEGINNER LAB | READ-ONLY
```

Run these commands in your Linux terminal. They are intended for **basic inspection** and do not modify system configuration.

1	2	3	4	5
`pwd`	`ls -l /`	`ls -ld`	`ls -l log`	`whoami`
where am I	file types	key dirs	find logs	who am I

STEP 1

Identify your current directory

```
$ pwd
```

Purpose: Prints the present working directory.

STEP 2

List directory contents with file types

```
$ ls -l /
```

Purpose: Lists entries in the root directory. Look at the first character of each permission string to distinguish regular files (`-`) from directories (`d`).

STEP 3

Inspect important directories

```
$ ls -ld /etc /var /tmp /home /boot
```

Purpose: Displays information about the directories themselves.

STEP 4

Locate log files

```
$ ls -l /var/log
```

Purpose: Lists available log files and subdirectories. Your permissions and Linux distribution determine what you can see.

STEP 5

Identify your account

```
$ whoami
$ id
```

Purpose: Shows your current username and user/group identity information.

// SAFETY NOTE

Do **not** delete, edit, or change permissions on system directories during this exercise.

07 SOC Interview Questions and Answers

// FLASHCARD 01

Q1. What is the root directory in Linux?

The root directory, represented by `/`, is the top-level directory from which the Linux filesystem hierarchy branches.

// FLASHCARD 02

Q2. What is the difference between `/` and `/root`?

`/` is the top-level filesystem directory. `/root` is the home directory of the root administrator account.

// FLASHCARD 03

Q3. Where would you look for Linux system logs?

I would start with `/var/log`, then identify the relevant log source and correlate its events with the alert. Some systems also store logs in a systemd journal rather than individual text files.

// FLASHCARD 04

Q4. What is the purpose of `/etc`?

It stores system-wide and application configuration files. During an investigation, I might inspect relevant configuration files for unauthorized changes.

// FLASHCARD 05

Q5. Why might a SOC analyst inspect `/tmp`?

Temporary directories can contain scripts or payloads created during suspicious activity. However, the presence of a file in `/tmp` alone does not prove malicious activity.

// FLASHCARD 06

Q6. How does Linux's directory structure differ from Windows?

Linux uses a unified hierarchy rooted at `/`, while Windows commonly uses drive letters such as `C:\` and `D:\`. Linux can still mount separate storage devices or filesystems within its unified hierarchy.

08 Quick Revision Checklist

REVISION PROGRESS 0 of 8

- [ ] Explain the difference between a file and a directory.
- [ ] Identify `/` and explain how it differs from `/root`.
- [ ] Recall the purpose of `/bin` and `/sbin`.
- [ ] Explain why `/etc` matters during investigations.
- [ ] Identify `/var/log` as a common location for logs.
- [ ] Explain how `/tmp` and `/home` can be relevant to an investigation.
- [ ] Describe the role of `/boot`.
- [ ] Use `pwd`, `ls -l`, and `whoami` in a Linux terminal.

LOGS
Where events are recorded
`/var/log`

CONFIG
Settings to check for changes
`/etc`

USER FILES
Downloads, scripts, activity
`/home`

TEMP ARTIFACTS
Scripts and payloads to validate
`/tmp`

`// KEY TAKEAWAY`

For an entry-level SOC Analyst, prioritize knowing **where to look**: `/var/log` for logs, `/etc` for configuration, `/home` for user files, and `/tmp` for temporary artifacts. These locations provide useful investigative context, but **findings must be validated and correlated before drawing conclusions**.

LINUX SECURITY // ENTRY-LEVEL SOC ANALYST PREPARATION

Securing Linux

SOC-Relevant Notes | Visual edition | Notion-ready study notes

```
analyst@linux-host: ~
$ sudo apt update
$ sudo apt upgrade
$ sudo ufw status
$ sudo ss -tulpn
$ sudo journalctl -p warning
```

8 MEASURES
essential hardening steps

CONTROL MAP
threat to evidence

SSH SCENARIO
detect to prevent

COMMANDS
for practice

INTERVIEW
six flashcards

CYAN course content **GREEN** SOC relevance / code **AMBER** interview point **RED** caution

01 Learning Objectives

// COURSE CONTENT
By the end of this topic, you should be able to:

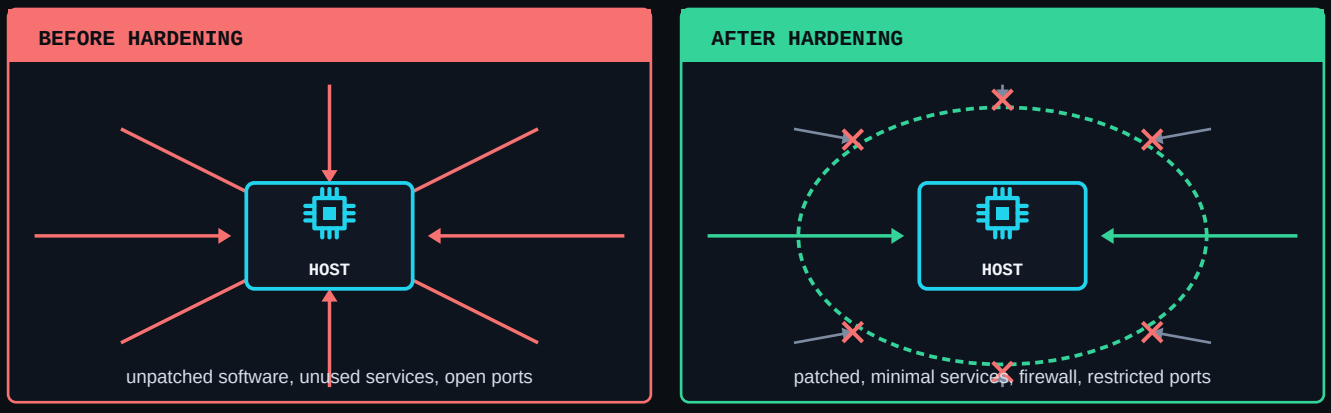
IDENTIFY
Identify essential security measures for protecting Linux systems.

IMPLEMENT
Explain how to implement each security measure.

REDUCE
Understand how Linux hardening reduces attack surfaces and security risks.

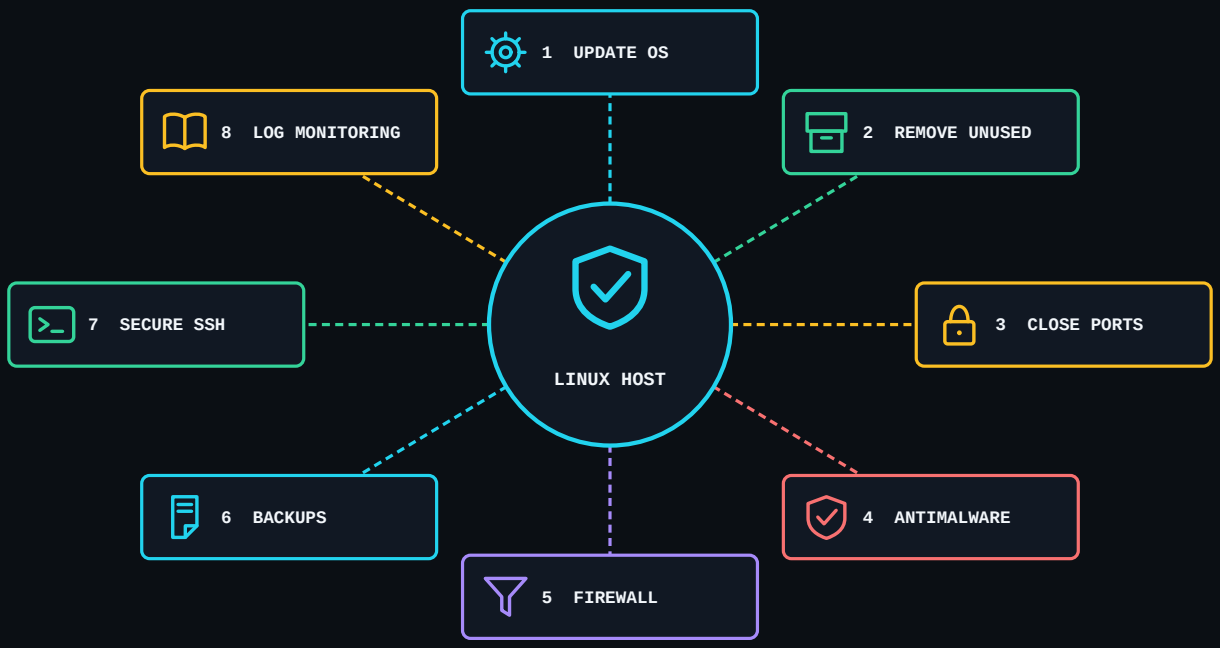
CONNECT
Connect Linux security controls to SOC monitoring, investigation, and incident response.

HOW HARDENING REDUCES THE ATTACK SURFACE



02 Essential Linux Security Measures

THE 8 MEASURES AROUND YOUR LINUX HOST



// MEASURE 01

1. Update the operating system

What: Install security patches and software updates.

How: On Ubuntu/Debian-based systems, run:

```
$ sudo apt update
$ sudo apt upgrade
```

Why: Patches fix known vulnerabilities that attackers could exploit.

SOC relevance: Investigate systems running vulnerable software and correlate exploitation alerts with patch status.

// MEASURE 02

2. Remove unused software and services

What: Remove unnecessary applications and disable services that are not required.

How: Package commands:

```
$ sudo apt remove package_name
$ sudo apt purge package_name
```

Why: Fewer installed applications and running services mean fewer potential attack paths.

SOC relevance: Investigate unexpected services, unauthorized software installations, and suspicious processes.

`remove`

Uninstalls the package but generally **leaves system-wide configuration files**.

`purge`

Also **removes package-managed configuration files**.

// MEASURE 03

3. Close unused network ports

What: Limit network access to services that are actually required.

How: Identify listening services and review firewall rules.

```
$ sudo ss -tulpn
```

Why: Unnecessary listening services can expose the system to unauthorized access.

SOC relevance: Investigate unexpected listening ports, suspicious network services, and unauthorized remote-access mechanisms.

// MEASURE 04

4. Install antimalware protection

What: Use reputable security software to detect and respond to malicious files and activity.

How: Deploy an appropriate endpoint protection or antimalware solution and keep its detection capabilities updated.

Why: Helps detect malware, malicious scripts, and other threats.

SOC relevance: Investigate malware detections, quarantine events, suspicious file hashes, and endpoint alerts.

// MEASURE 05

5. Enable a firewall

What: Control permitted inbound and outbound network traffic.

How: Ubuntu commonly supports UFW (Uncomplicated Firewall).

```
$ sudo ufw status
```

Why: Restricts unauthorized network connections and reduces exposure.

SOC relevance: Review firewall rules and logs when investigating suspicious connections or attempted access.

// MEASURE 06

6. Set up regular backups

What: Maintain recoverable copies of important data and system configurations.

How: Schedule backups and store copies in separate, access-controlled locations, including offline or immutable storage where appropriate. **Test restoration regularly.**

Why: Backups support recovery from ransomware, accidental deletion, and data corruption.

SOC relevance: During ransomware incidents, verify backup integrity and follow approved recovery procedures.

// MEASURE 07

7. Secure SSH access

What: Harden Secure Shell (SSH), which provides remote command-line access.

How: Disable direct root login where appropriate, prefer strong key-based authentication, and enable MFA where supported.

Why: Reduces the risk of brute-force attacks and unauthorized remote access.

SOC relevance: Investigate repeated failed logins, unfamiliar source IPs, unusual successful logins, and unexpected SSH configuration changes.

// MEASURE 08

8. Monitor system logs

What: Review system and application events for suspicious activity.

How: Inspect relevant log files and use system logging or centralized monitoring tools.

```
$ sudo journalctl -p warning
```

Why: Log monitoring helps identify suspicious activity early and supports incident investigation.

SOC relevance: Correlate authentication events, service failures, privilege changes, and endpoint or network alerts.

03 SOC Security Control Mapping

// COURSE CONTENT

Each control reduces a specific risk, and each leaves **evidence a SOC analyst can investigate** when it fails or is bypassed.

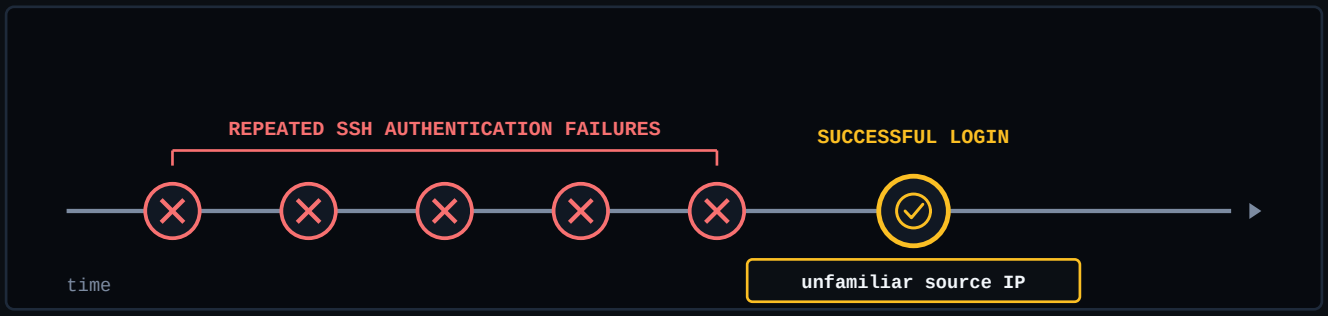
SECURITY MEASURE	THREAT OR RISK REDUCED	SOC ANALYST MIGHT INVESTIGATE
OS updates	Exploitation of known vulnerabilities	Vulnerable versions and exploitation attempts
Remove unused software	Unnecessary attack surface	Unauthorized packages and services
Close unused ports	Unwanted network exposure	Unexpected listeners and connections
Antimalware	Malicious software	Malware detections and suspicious files
Firewall	Unauthorized network access	Blocked connections and rule changes
Backups	Data loss and ransomware impact	Backup failures, tampering, recovery readiness
SSH hardening	Brute-force and remote-access abuse	Failed logins and unusual successful logins
Log monitoring	Delayed threat detection	Anomalies, event sequences, and indicators of compromise

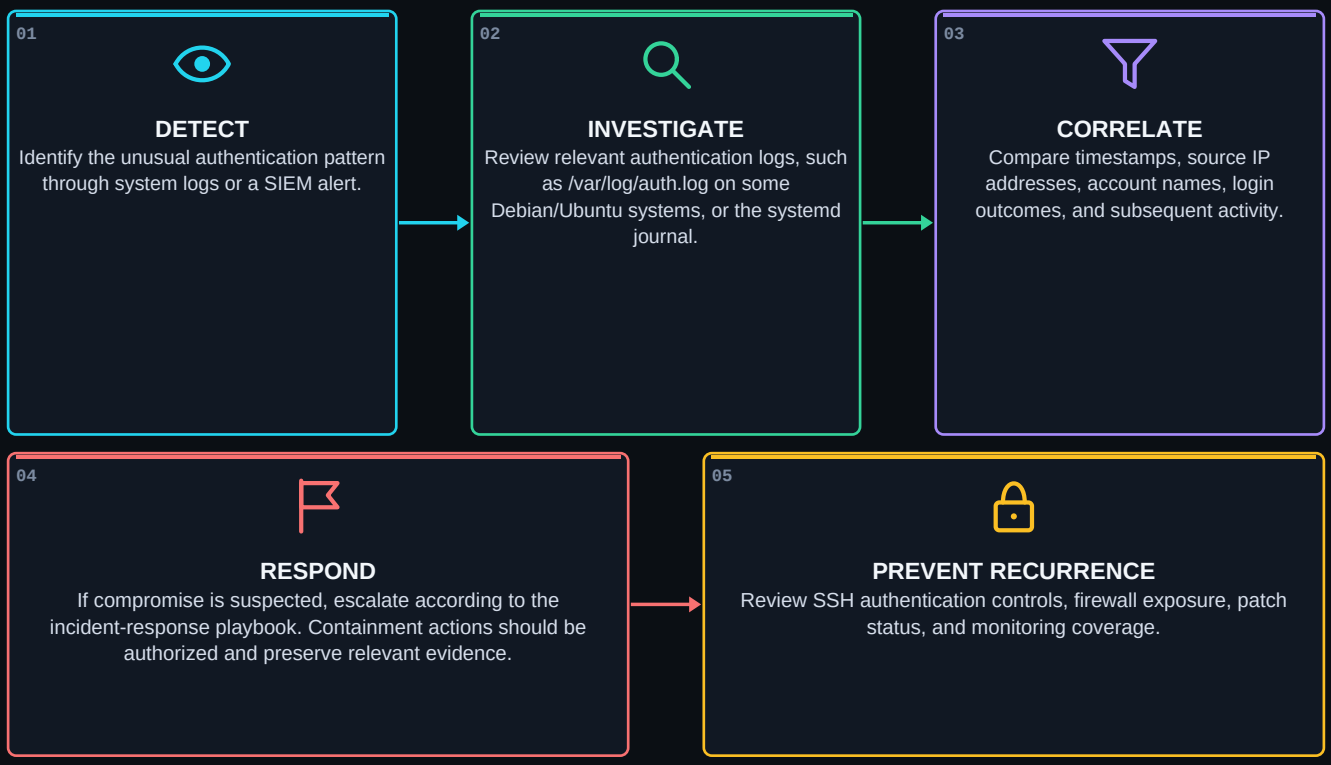
04 Practical SOC Investigation Scenario

// SCENARIO

A Linux server generates **repeated SSH authentication failures, followed by a successful login from an unfamiliar IP address**.

EVENT TIMELINE





INVESTIGATE: QUERY THE SSH SERVICE LOGS

```
$ sudo journalctl -u ssh
```

// NOTE
The SSH service name may differ by distribution.

05 Important Commands for Practice

COMMAND	PURPOSE	SECURITY RELEVANCE
`sudo apt update`	Refresh package metadata	Prepare to check for updates
`sudo apt upgrade`	Upgrade installed packages	Apply available package updates
`sudo ss -tulpn`	Show listening sockets	Identify exposed services
`sudo ufw status`	Show UFW status and rules	Review host firewall configuration
`whoami`	Show current username	Establish current account context
`id`	Show user and group IDs	Check group membership and privilege context
`sudo journalctl -p warning`	Display warning-and-higher-priority journal entries	Review selected system events
`sudo journalctl -u ssh`	Query SSH service logs	Investigate remote-access activity

// SAFETY NOTE
Use administrative commands **only on systems you are authorized to manage**. Before changing firewall rules, SSH settings, or packages on a remote server, confirm that you will **not lock yourself out or disrupt production services**.

06 SOC Interview Questions and Answers

COMPLEMENTARY PROTECTIONS

Firewall
Controls network traffic according to configured rules.

Antimalware
Detects or blocks malicious files and activity.

// FLASHCARD 01

Q1. Why is patch management important in Linux security?

Patch management fixes known vulnerabilities, reducing the likelihood that attackers can exploit outdated software.

// FLASHCARD 02

Q2. Why should unused services be disabled?

Unused services increase the attack surface and may expose unnecessary network-accessible functionality.

// FLASHCARD 03

Q3. What is the difference between a firewall and antimalware software?

A firewall controls network traffic according to configured rules. Antimalware software detects or blocks malicious files and activity. They provide different, complementary protections.

// FLASHCARD 04

Q4. Why is SSH hardening important?

SSH is commonly used for remote administration. Strong authentication, restricted root access, and appropriate access controls reduce the risk of unauthorized remote access.

// FLASHCARD 05

Q5. How do system logs help SOC analysts?

Logs provide evidence of system and application events. Analysts correlate them with other telemetry to identify suspicious activity, reconstruct timelines, and support incident response.

// FLASHCARD 06

Q6. How do backups help during a ransomware incident?

Verified backups can support restoration of affected systems and data. Isolated or immutable copies help reduce the risk that attackers can destroy recovery options.

07 Quick Revision Checklist

REVISION PROGRESS 0 / 8

- [ ] Explain why security patches must be applied regularly.
- [ ] Describe how removing unused software reduces attack surface.
- [ ] Identify listening services using `ss`.
- [ ] Explain the different roles of firewalls and antimalware.
- [ ] Describe a secure SSH configuration strategy.
- [ ] Explain why backups should be isolated and restoration-tested.
- [ ] Identify Linux logs relevant to SSH investigations.
- [ ] Describe how a SOC analyst detects, investigates, and escalates suspicious access.

CONTROLS AT A GLANCE

PATCHING	`apt update` `apt upgrade`
MINIMIZE SERVICES	`apt remove` `apt purge`
NETWORK ACCESS	`ss -tulpn` `ufw status`
ANTIMALWARE	`detect + quarantine`
BACKUPS	`isolated` `restore-tested`
SSH AUTH	`key-based` `no root login` `MFA`
LOG MONITORING	`journalctl -p warning` `journalctl -u ssh`

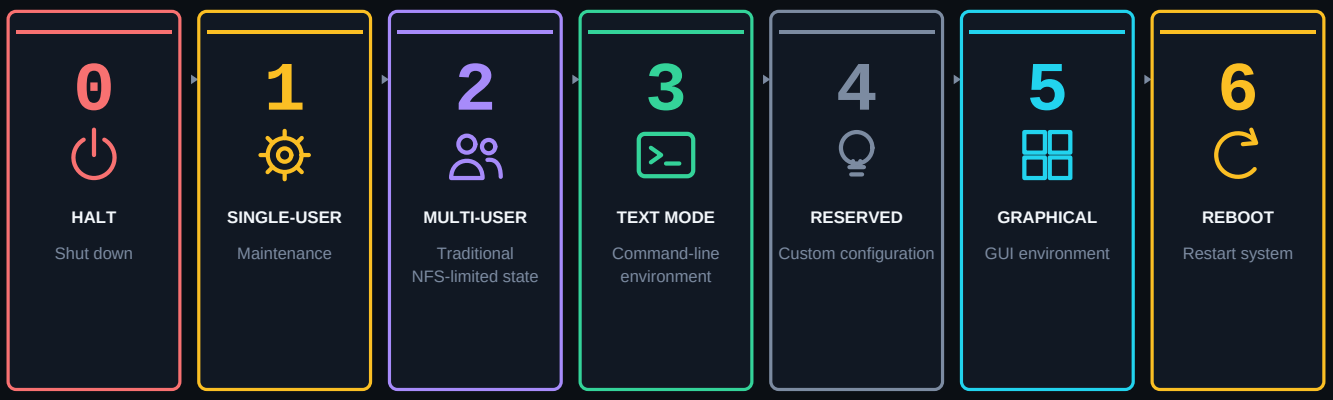
// KEY TAKEAWAY

Linux security relies on **multiple complementary controls**: patching, minimizing installed services, restricting network access, antimalware, backups, secure SSH authentication, and log monitoring. For a SOC analyst, the essential skill is understanding **both how these controls prevent attacks and what evidence to investigate when a control fails**.

Linux Run Levels

SOC-Relevant Notes | Visual edition | Notion-ready notes

```
analyst@linux-host: ~
$ systemctl get-default
$ systemctl list-units --type=target
$ uptime
$ who -b
$ last -x
```



CYAN course content **GREEN** SOC relevance / code **AMBER** interview point **RED** caution

01 Learning Objectives

DEFINE
Define Linux run levels and explain their purpose.

IDENTIFY
Identify run levels 0-6 and their operating states.

UNDERSTAND
Understand how run levels affect system services and user access.

CONNECT
Connect Linux operating states to system administration and SOC investigations.

02 What Are Linux Run Levels?

// DEFINITION
A run level is a **defined operating state** traditionally used by Linux systems to determine which services and processes should run.

// COURSE CONTENT
In traditional **SysV init** systems, run levels are identified by numbers from **0 to 6**. Each level represents a different system state, such as shutting down, performing maintenance, running in text mode, or providing a graphical interface.

WHO MANAGES RUN LEVELS?

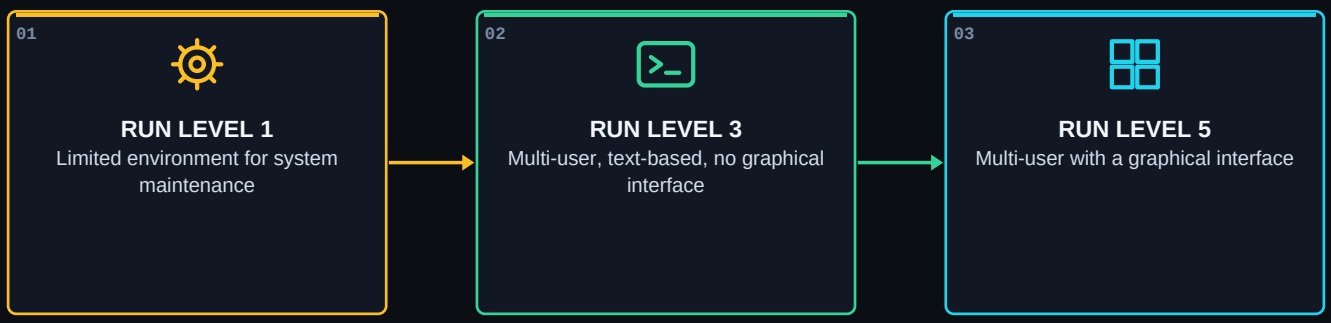


03 Linux Run Levels Explained

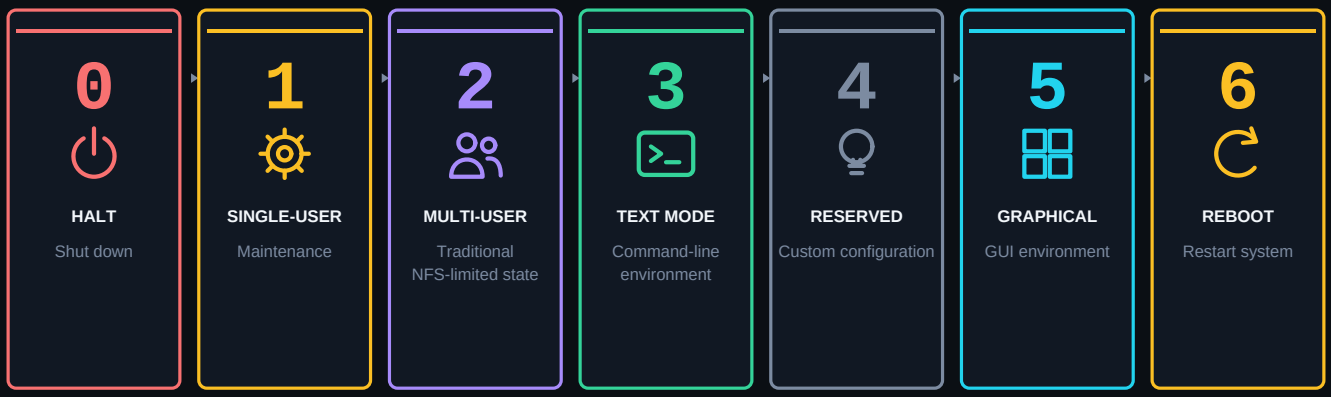
LEVEL	NAME	DESCRIPTION	SOC RELEVANCE
0	**Halt**	Stops system operations and shuts down the system.	Investigate unexpected shutdowns or availability disruptions.
1	**Single-user mode**	Provides a limited environment for system maintenance.	Understand recovery and maintenance access risks.
2	**Multi-user without NFS***	Allows multiple users without network file system services in the traditional model.	Understand service availability and system configuration.
3	**Multi-user, text-based**	Supports multiple users without a graphical interface.	Common concept for server administration and command-line investigations.
4	**Undefined/reserved**	Traditionally available for custom configuration.	A customized state may require investigation if unexpected.
5	**Multi-user, graphical**	Supports multiple users with a graphical interface.	Useful when understanding workstation operating states.
6	**Reboot**	Restarts the system.	Investigate unexpected reboot events and possible service interruptions.

// NOTE
*NFS means **Network File System**. These descriptions reflect the traditional run-level model; exact behavior depends on the init system and distribution.

TYPICAL PROGRESSION: LIMITED TO FULL ENVIRONMENT



04 Visual Memory Aid

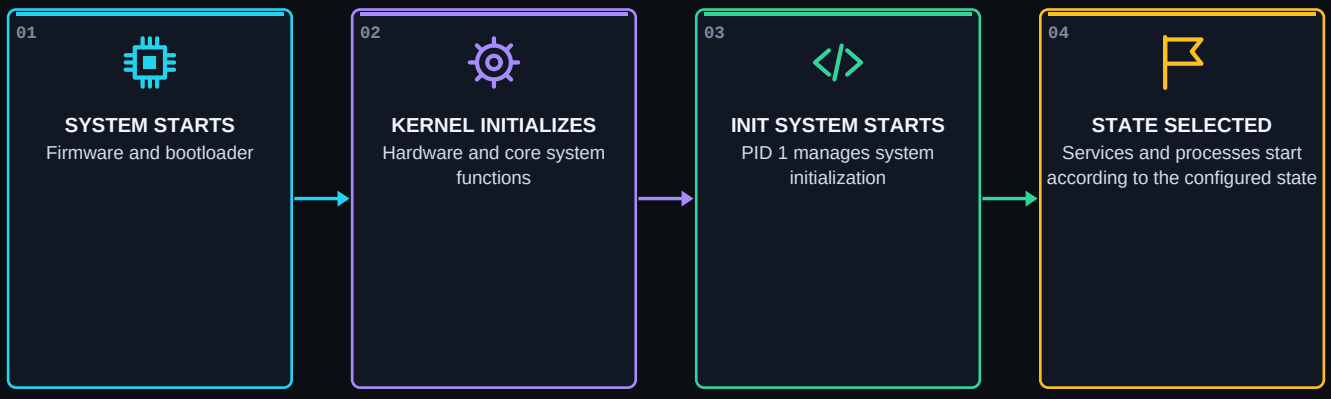


MEMORY TIP

0 = Halt	1 = Maintenance	3 = Text	5 = GUI	6 = Reboot

05 How Run Levels Work

THE TRADITIONAL LINUX BOOT SEQUENCE



// IMPORTANT MODERN LINUX DISTINCTION

Many current Linux distributions, including Ubuntu, use **systemd** rather than traditional SysV init. Systemd uses **targets** that provide functionality similar to traditional run levels.

SYSV RUN LEVEL vs SYSTEMD TARGET

TRADITIONAL RUN LEVEL (SYSV INIT)		COMMON SYSTEMD EQUIVALENT
0	Halt	poweroff.target
1	Single-user	rescue.target
2	Multi-user w/o NFS	*no universal equivalent*
3	Multi-user, text	multi-user.target
4	Undefined/reserved	*no universal equivalent*
5	Multi-user, graphical	graphical.target
6	Reboot	reboot.target

// NOTE
Run levels **2** and **4** do **not have universal equivalents** because their traditional meanings can vary.

USEFUL COMMANDS ON SYSTEMD-BASED SYSTEMS

```
# View the default boot target
$ systemctl get-default

# View the current system state
$ systemctl list-units --type=target

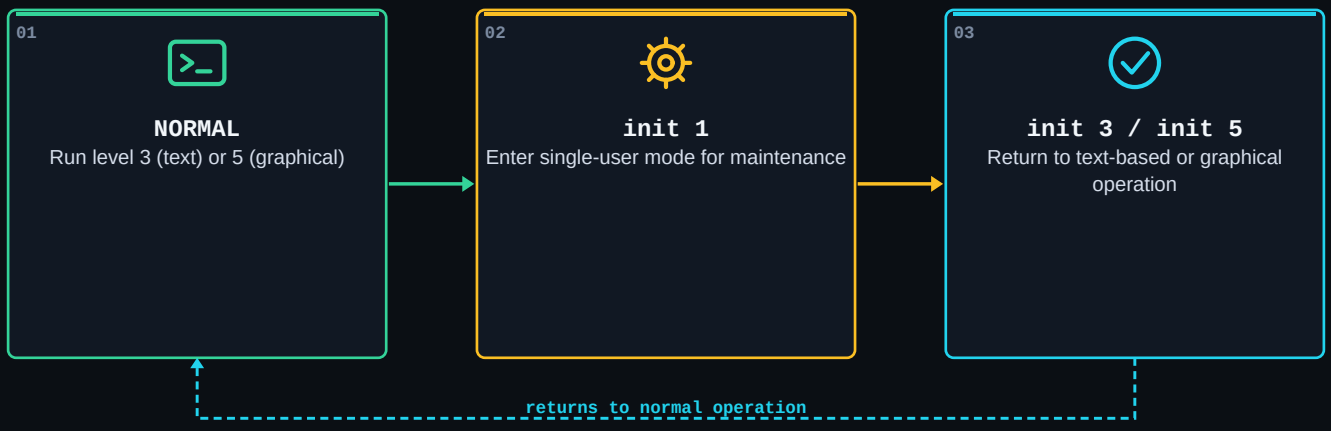
# View the system's uptime and recent boot
$ uptime
$ who -b
```

// SAFE TO RUN
These commands **inspect** system state; they do not change the run level or target.

06 Switching Run Levels: Maintenance Example

// COURSE CONTENT

The supplied reading describes using `init 1` to enter **single-user mode** and `init 3` or `init 5` to return to text-based or graphical operation. Example from a traditional SysV init system:




```
$ sudo init 1

# To return to a normal operating state:
$ sudo init 3
# or:
$ sudo init 5
```

// CAUTION

These commands can **stop services, disconnect users, and interrupt workloads**. On modern systemd-based systems, the corresponding maintenance workflow differs. Do not experiment with changing system state on a production machine or remote server **without an approved maintenance plan and recovery access**.

// ALSO NOTE

Single-user or rescue mode is **not automatically a secure environment**. Its actual access restrictions depend on system configuration, and it may provide **powerful administrative access**.

07 SOC Relevance: Run Levels and System States

// SOC RELEVANCE

A SOC analyst may not routinely change run levels, but understanding system states **helps with incident investigation**.

EVIDENCE SOURCES

BOOT HISTORY	`last -x`
SYSTEM LOGS	`journalctl`
ACTIVE TARGET	`systemctl get-default` `list-units --type=target`
UPTIME + LAST BOOT	`uptime` `who -b`

// SCENARIO 1

Unexpected shutdown or reboot

A server unexpectedly restarts during business hours.

- Review boot history with `last -x`.
- Check relevant system logs using `journalctl`.
- Correlate the restart with maintenance records, crashes, or suspicious activity.

Potential concern: service disruption or availability impact.

// SCENARIO 2

Unexpected maintenance mode

A server boots into a recovery or rescue environment unexpectedly.

- Confirm the system's current state and recent boot events.
- Check whether authorized maintenance occurred.
- Investigate unusual administrative access or configuration changes.

Potential concern: unauthorized access or system recovery after a failure.

// SCENARIO 3

Server running without expected services

A server is online, but its normal applications are unavailable.

- Check the active target and service states.
- Review failed services and recent logs.
- Determine whether a configuration change or system failure explains the behavior.

Potential concern: degraded service availability or unauthorized system changes.

08 SOC Interview Questions and Answers

// FLASHCARD 01

Q1. What is a Linux run level?

A run level is a traditional operating state that determines which services and processes should be active.

// FLASHCARD 02

Q2. What is run level 1 used for?

It is traditionally associated with single-user or maintenance mode, where an administrator can perform recovery and maintenance tasks.

// FLASHCARD 03

Q3. What is the difference between run levels 3 and 5?

Run level 3 traditionally provides multi-user text-based operation, while run level 5 provides multi-user operation with a graphical interface.

// FLASHCARD 04

Q4. What do run levels 0 and 6 represent?

Run level 0 represents system halt or shutdown, and run level 6 represents reboot.

// FLASHCARD 05

Q5. Are traditional run levels used by every modern Linux distribution?

No. Many modern distributions use systemd targets, which provide equivalent operating states. The exact implementation depends on the distribution.

// FLASHCARD 06

Q6. How can run-level knowledge help a SOC analyst?

It helps analysts understand system availability, unexpected shutdowns, maintenance or recovery states, and situations where expected services are not running.

WHAT TO RECOGNIZE, THEN VALIDATE

UNEXPECTED REBOOTS
Check boot history and logs

RESCUE STATES
Confirm authorized maintenance

MISSING SERVICES
Check target and service status

// KEY TAKEAWAY

Traditional Linux run levels define **operating states and the services associated with them**. For SOC work, focus on recognizing **unexpected reboots, rescue states, and missing services**, then validating the cause using **system logs, boot history, and service status**.